Dockerized Contact Application with Automated MongoDB Backup, Monitoring, and Disaster Recovery

Project Report

---

1. Project Overview

This project demonstrates the deployment and operation of a production-style, containerized Contact Application on an Amazon EC2 instance. The application consists of a frontend, backend, and MongoDB database running as separate Docker containers.

The project was designed not only to deploy the application but also to implement important DevOps and operational practices, including:

- Multi-container application deployment using Docker Compose
- Docker network segmentation
- Persistent MongoDB storage using a bind mount
- Automated MongoDB backups
- Backup storage in Amazon S3
- Automated backup scheduling using Cron
- Backup retention and cleanup
- Serverless backup monitoring using AWS Lambda
- Automated alerts using Amazon SNS
- Disaster recovery testing using MongoDB restore operations

The final implementation demonstrates a complete lifecycle from application deployment to data protection, monitoring, alerting, and recovery.

---

2. Problem Statement

Modern applications require more than simply running containers successfully. Application data must be protected against accidental deletion, container failures, infrastructure problems, and operational mistakes.

MongoDB data stored inside a container can be lost if persistence is not configured correctly. Additionally, creating backups manually is unreliable and does not provide a complete disaster recovery strategy.

This project addresses these challenges by implementing a containerized application architecture with:

1. Persistent database storage
2. Automated database backups
3. Off-server backup storage in Amazon S3
4. Scheduled backup execution
5. Backup age monitoring
6. Automated alert notifications
7. Tested disaster recovery procedures

---

3. Project Objectives

The main objectives of this project were:

- Deploy a multi-container Contact Application using Docker Compose.
- Separate the frontend and database communication layers using Docker networks.
- Persist MongoDB data outside the container lifecycle.
- Automate MongoDB database backups.
- Store backup archives securely in Amazon S3.
- Schedule daily backup execution using Cron.
- Implement backup retention and cleanup.
- Monitor whether recent backups are available.
- Send automated alerts when a backup becomes outdated or missing.
- Perform a disaster recovery test by deleting and restoring application data.

---

4. Technology Stack

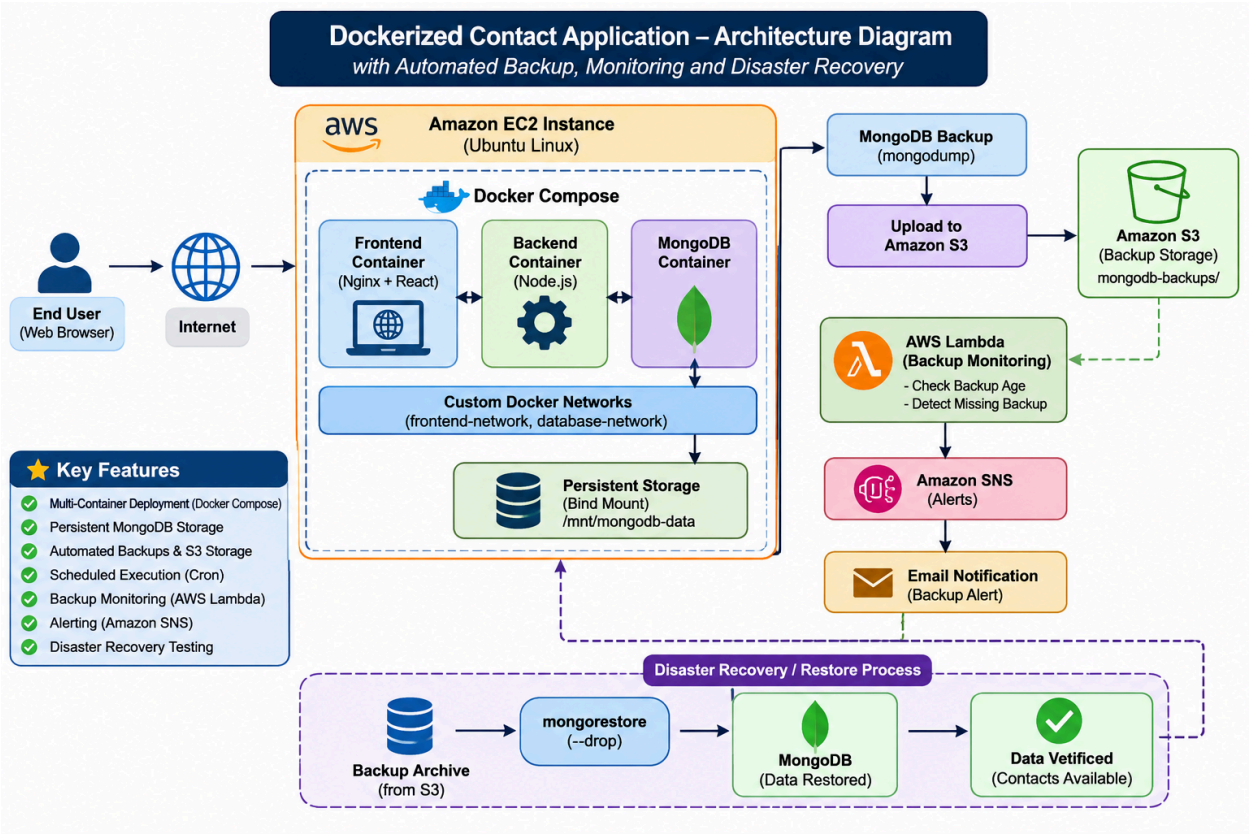
Technology	Purpose

Amazon EC2	Application hosting infrastructure
Ubuntu Linux	Server operating system
Docker	Containerization platform
Docker Compose	Multi-container application orchestration
React / Frontend	User interface
Node.js	Backend application
MongoDB	Application database
Nginx	Frontend web server
Amazon S3	Remote backup storage
AWS Lambda	Backup monitoring logic
Amazon SNS	Alert notification service
Cron	Automated backup scheduling
mongodump	MongoDB backup utility
mongorestore	MongoDB disaster recovery utility

---

5. High-Level Architecture

The application follows a multi-container architecture.



---

6. Multi-Container Application Deployment

The Contact Application was deployed using Docker Compose. The application consists of three primary containers:

- Contact Frontend
- Contact Backend
- MongoDB Database

Each component runs independently inside its own container.

This approach provides:

- Application isolation
- Independent service management
- Simplified deployment
- Improved scalability
- Easier networking configuration

Application Containers Running

The Docker Compose environment was verified using the Docker Compose status command. All application services were successfully running.

Docker Compose Application Services Running on Amazon EC2



```
ubuntu@ip-172-31-6-0:~/docker-contact-app$ sudo docker compose ps
NAME              IMAGE                          COMMAND                  SERVICE    CREATED      STATUS              PORTS
contact-backend   docker-contact-app-backend     "docker-entrypoint.s…"   backend    2 days ago   Up 44 seconds       5000/tcp
contact-frontend  docker-contact-app-frontend    "/docker-entrypoint.…"   frontend   2 days ago   Up 45 seconds       0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
contact-mongodb   mongo:8.2                      "docker-entrypoint.s…"   mongodb    2 days ago   Up About a minute   27017/tcp
```

The successful container status confirms that the frontend, backend, and MongoDB services were running correctly.

---

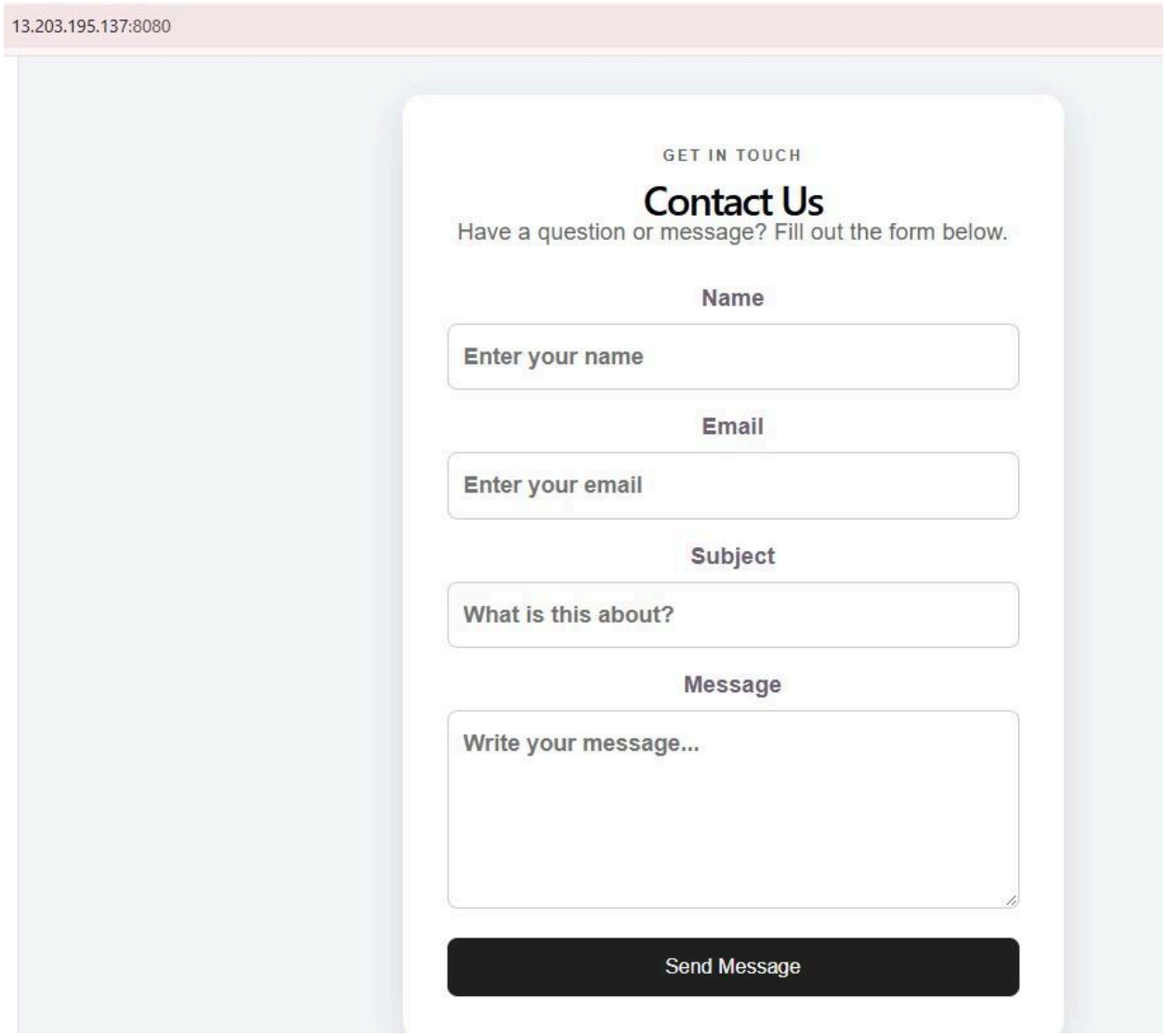
7. Application Accessibility

The frontend application was successfully accessed through the Amazon EC2 public IP address using port `8080`.

The successful browser access confirms that:

- The EC2 instance was reachable.
- Docker port publishing was configured correctly.
- The frontend container was running successfully.
- The application was accessible externally.

Dockerized Contact Application Successfully Accessible Through the Amazon EC2 Public IP



---

8. MongoDB Database Verification

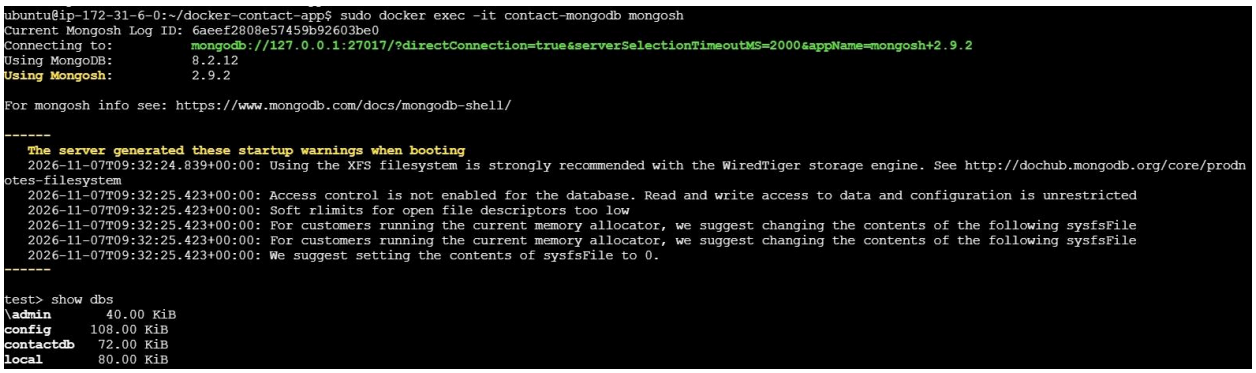
The MongoDB container was accessed directly using `mongosh` to verify that the application database existed and that application data was successfully stored.

The following database was used:

```
None
contactdb
```

The application stores contact form submissions inside the `contacts` collection.

Successful Connection to MongoDB Running Inside the Docker Container



The database verification confirmed that the MongoDB service was running successfully inside the Docker environment.

---

9. Application Data Storage Verification

The contact records stored through the application were directly verified inside MongoDB.

The records contained information such as:

- Name
- Email
- Subject
- Message
- Creation timestamp

Contact Application Data Successfully Stored and Verified in MongoDB

```
test> use contactdb
switched to db contactdb
contactdb>

contactdb> show collections
contacts
contactdb> db.contacts.find()
[
  {
    _id: ObjectId('6aeb4483ff19c29b8b07c522'),
    name: 'Nandan. K. R',
    email: 'nandanmayura600@gmail.com',
    subject: 'test',
    message: 'testing multi container\n',
    createdAt: ISODate('2026-11-04T14:43:15.613Z'),
    updatedAt: ISODate('2026-11-04T14:43:15.613Z'),
    __v: 0
  },
  {
    _id: ObjectId('6aeef20467390c18ba199ae0'),
    name: 'user2',
    email: 'user2@gmailcom',
    subject: 'devops',
    message: 'testing multi container',
    createdAt: ISODate('2026-11-07T09:40:20.755Z'),
    updatedAt: ISODate('2026-11-07T09:40:20.755Z'),
    __v: 0
  }
]
```

---

10. Docker Network Architecture

The application uses custom Docker bridge networks instead of placing all services on a single network.

The implemented networks include:

```
None
docker-contact-app_frontend-network
```

```
docker-contact-app_database-network
```

Custom Docker Bridge Networks Created for Application Communication

```
ubuntu@ip-172-31-6-0:~/docker-contact-app$ sudo docker network ls
NETWORK ID     NAME                                      DRIVER    SCOPE
1bd1f7eb9a0f   bridge                                    bridge    local
050f5d11b150   docker-contact-app_database-network       bridge    local
90aca9d5de98   docker-contact-app_frontend-network       bridge    local
bfcf8daf60e4   host                                      host      local
f2b9ee60abb5   none                                      null      local
ubuntu@ip-172-31-6-0:~/docker-contact-app$
```

This architecture improves container isolation and follows the principle of network segmentation.

---

11. Container Network Segmentation

The containers were configured with controlled network access.

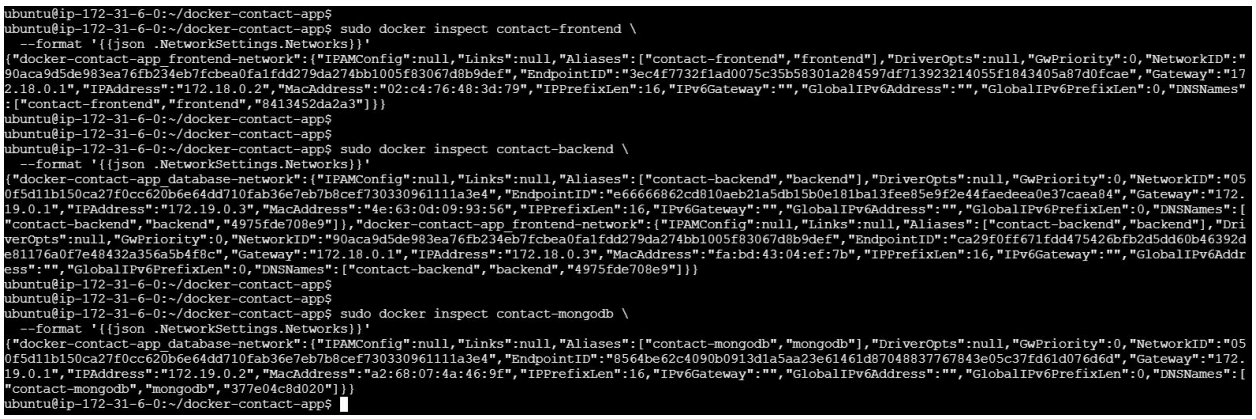
The network architecture follows this pattern:

```
None
Contact Frontend
        |
        | frontend-network
        ▼
Contact Backend
        |
        | database-network
        ▼
MongoDB
```

The backend container acts as the communication layer between the frontend and the database.

The frontend does not directly communicate with MongoDB.

Docker Network Segmentation Showing Backend Communication Between Frontend and MongoDB



This design improves security and follows a layered application architecture.

---

12. Persistent MongoDB Storage

MongoDB data was configured to persist outside the MongoDB container using a bind mount.

The MongoDB database directory:

```
None
/data/db
```

is mapped to the Amazon EC2 host location:

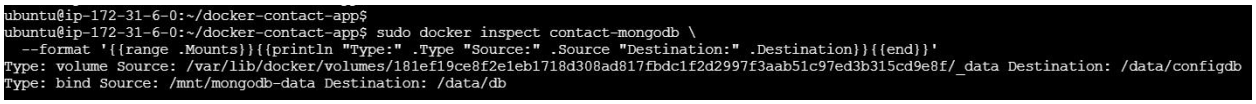
```
None
/mnt/mongodb-data
```

The storage architecture is:

```
None
MongoDB Container
        |
        | /data/db
```

```
        ▼
Docker Bind Mount
        |
        ▼
EC2 Host Storage
/mnt/mongodb-data
```

Persistent MongoDB Storage Using an Amazon EC2 Host Bind Mount



This configuration ensures that database data can persist independently of the MongoDB container lifecycle.

---

13. MongoDB Database Files on Persistent Storage

The EC2 host storage directory was inspected directly to verify that MongoDB database files existed outside the container.

The directory contained MongoDB WiredTiger storage files, including:

- `WiredTiger`
- `WiredTiger.wt`
- Collection files
- Index files
- Journal files
- Storage metadata

MongoDB WiredTiger Database Files Persistently Stored on the Amazon EC2 Host

```
ubuntu@ip-172-31-6-0:~/docker-contact-app$ sudo ls -lh /mnt/mongodb-data
total 516K
-rw------- 1 dnsmasq systemd-journal   50 Nov  4 14:30 WiredTiger
-rw------- 1 dnsmasq systemd-journal   21 Nov  4 14:30 WiredTiger.lock
-rw------- 1 dnsmasq systemd-journal 1.5K Nov  7 09:55 WiredTiger.turtle
-rw------- 1 dnsmasq systemd-journal  76K Nov  7 09:55 WiredTiger.wt
-rw------- 1 dnsmasq systemd-journal 4.0K Nov  7 09:32 WiredTigerHS.wt
-rw------- 1 dnsmasq systemd-journal  36K Nov  7 09:32 _mdb_catalog.wt
drwx------ 3 dnsmasq systemd-journal 4.0K Nov  7 09:32 _tmp
-rw------- 1 dnsmasq systemd-journal  32K Nov  7 09:48 collection-20c0376f-9b0a-44ca-8528-f148d056f4be.wt
-rw------- 1 dnsmasq systemd-journal  36K Nov  7 09:40 collection-31b55337-f468-448d-a0dd-a5296d531e48.wt
-rw------- 1 dnsmasq systemd-journal  44K Nov  7 09:33 collection-86d3a870-7f3b-44c8-9348-63beb618894a.wt
-rw------- 1 dnsmasq systemd-journal  20K Nov  7 09:32 collection-b855dd08-db20-4835-a486-835bb912fc75.wt
drwxr-xr-x 2 dnsmasq root            4.0K Nov  4 14:10 db
drwx------ 2 dnsmasq systemd-journal 4.0K Nov  7 09:55 diagnostic.data
-rw------- 1 dnsmasq systemd-journal  32K Nov  7 09:48 index-6ca9d264-cf8f-4298-81cc-8cb9b1f2d310.wt
-rw------- 1 dnsmasq systemd-journal  36K Nov  7 09:40 index-7715eec8-9420-4043-b812-4d5448d9507a.wt
-rw------- 1 dnsmasq systemd-journal  20K Nov  7 09:32 index-92c05684-df97-4f75-b44c-80e45d800567.wt
-rw------- 1 dnsmasq systemd-journal  36K Nov  7 09:42 index-aa1d43e2-56f0-423f-adf1-5cd78b304ced.wt
-rw------- 1 dnsmasq systemd-journal  36K Nov  7 09:33 index-c9590ef3-2d07-4d70-9905-f27bca648029.wt
drwx------ 2 dnsmasq systemd-journal 4.0K Nov  7 09:32 journal
drwx------ 2 dnsmasq root             16K Nov  4 14:02 lost+found
-rw------- 1 dnsmasq systemd-journal    2 Nov  7 09:32 mongod.lock
-rw------- 1 dnsmasq systemd-journal  32K Nov  7 09:44 sizeStorer.wt
-rw------- 1 dnsmasq systemd-journal  114 Nov  4 14:30 storage.bson
```

This confirms that the MongoDB data is stored persistently on the host.

---

14. Automated MongoDB Backup

A MongoDB backup process was implemented using MongoDB backup utilities.

The backup process performs the following operations:

```
None
MongoDB Database
        ↓
mongodump
        ↓
Backup Archive Created
        ↓
Stored Locally
        ↓
Uploaded to Amazon S3
        ↓
Retention Cleanup
```

The backup process successfully dumped the `contactdb.contacts` collection and created a backup archive.

Successful MongoDB Backup Creation, Amazon S3 Upload, and Local Backup Retention Cleanup



The successful output confirmed:

- MongoDB data was dumped successfully.
- A backup archive was created.
- The backup was copied to the local backup directory.
- The backup was uploaded to Amazon S3.
- Local retention cleanup was performed.

---

15. Amazon S3 Backup Storage

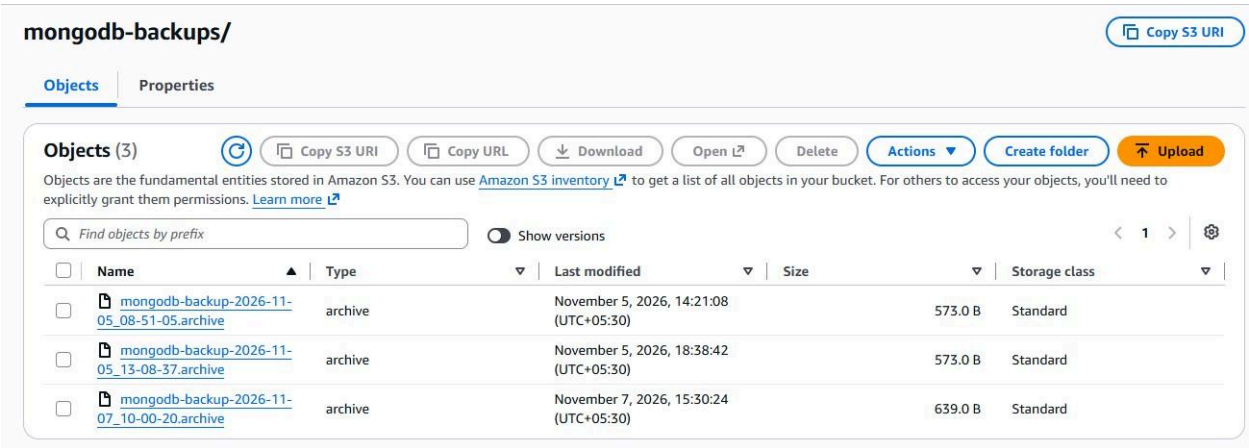
MongoDB backup archives were stored in an Amazon S3 bucket.

The backup objects were organized under the MongoDB backup prefix:

```
None
mongodb-backups/
```

Multiple backup archives were visible in Amazon S3, demonstrating repeated successful backup operations.

MongoDB Backup Archives Successfully Stored in the Amazon S3 Bucket



Storing backups in Amazon S3 provides an additional layer of protection because the backup data exists separately from the EC2 instance and Docker environment.

---

16. Automated Backup Scheduling

The MongoDB backup process was automated using Cron.

The configured Cron job executes the backup process daily at the scheduled time.

The automation workflow is:

```
None
Scheduled Time
      ↓
Cron Job
      ↓
backup-mongodb.sh
      ↓
MongoDB Backup
      ↓
Amazon S3 Upload
```

Figure 12: Automated Daily MongoDB Backup Scheduling Using Cron

```
ubuntu@ip-172-31-6-0:~/docker-contact-app/scripts$
ubuntu@ip-172-31-6-0:~/docker-contact-app/scripts$ crontab -l
0 2 * * * /home/ubuntu/docker-contact-app/scripts/backup-mongodb.sh >> /home/ubuntu/mongodb-backups/backup.log 2>&1
ubuntu@ip-172-31-6-0:~/docker-contact-app/scripts$
```

This eliminates the need for manual backup execution and improves operational reliability.

---

17. Backup Retention and Cleanup

The backup process includes a retention mechanism for local backup files.

Older local backups are automatically cleaned according to the configured retention period.

This helps:

- Prevent unnecessary disk consumption.
- Maintain controlled backup storage.
- Automatically remove outdated local backups.

The retention cleanup was verified as part of the successful backup execution process.

---

18. Backup Monitoring Using AWS Lambda

Creating backups is not sufficient if the system cannot detect when backups stop occurring.

To solve this problem, an AWS Lambda function was implemented to monitor the age of the latest MongoDB backup.

The Lambda monitoring process performs the following workflow:

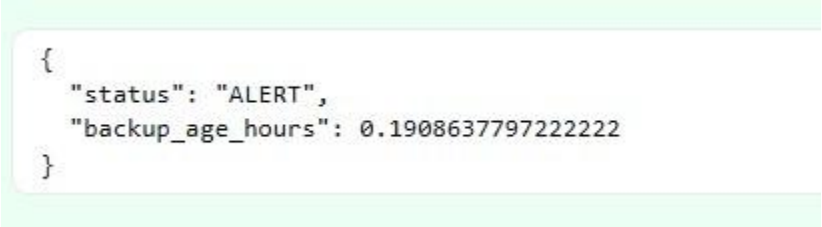
```
None
Amazon S3
      ↓
Check Latest Backup
      ↓
Calculate Backup Age
      ↓
Compare Against Allowed Threshold
      ↓
```

```
Healthy / Alert
```

During testing, the monitoring threshold was temporarily configured to trigger an alert.

The Lambda execution successfully detected an outdated backup condition.

AWS Lambda Successfully Detecting an Outdated MongoDB Backup



The result confirmed that the monitoring logic could identify a backup that exceeded the configured acceptable age.

---

19. Automated Alerting Using Amazon SNS

When the backup monitoring system detects an outdated or missing backup, an automated notification is sent using Amazon SNS.

The alerting workflow is:

```
None
Amazon S3 Backup
        ↓
AWS Lambda Monitoring
        ↓
Backup Age Validation
        ↓
Alert Condition Detected
        ↓
Amazon SNS
```

```
        ↓
Email Notification
```

The monitoring alert was successfully delivered through email.

Automated MongoDB Backup-Missing Alert Successfully Delivered Through Amazon SNS



This confirms that administrators can be notified automatically when backup protection is no longer functioning as expected.

---

20. Disaster Recovery Testing

A backup system is only useful if the data can be successfully restored.

Therefore, a disaster recovery test was performed.

A valid MongoDB backup archive was created using `mongodump`.

The backup archive was then used for restoration testing.

The recovery process was:

```
None
Application Data
      ↓
Create Backup Archive
      ↓
```

```
Delete Existing Data
        ↓
Restore Backup
        ↓
Verify Recovered Data
```

---

21. MongoDB Data Restoration

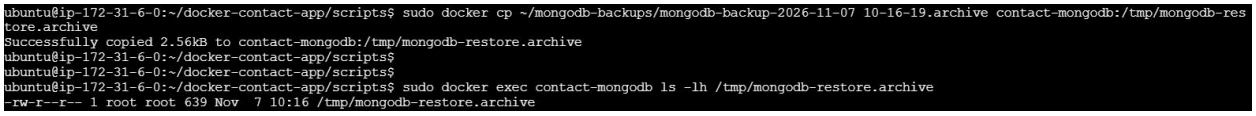
The MongoDB backup was restored using `mongorestore`.

The restore operation successfully processed the `contactdb.contacts` collection.

The restore output confirmed:

```
None
2 document(s) restored successfully
0 document(s) failed to restore
```

Successful MongoDB Disaster Recovery Using mongorestore



The restore process successfully recreated the contact collection from the backup archive.

---

22. Restored Data Verification

After completing the restore operation, MongoDB was accessed directly to verify the recovered contact records.

The restored database contained the expected contact documents.

Verification of Successfully Recovered Contact Data After MongoDB Restore

```
ubuntu@ip-172-31-6-0:~$ sudo docker exec -it contact-mongodb mongosh --eval '
use("contactdb");
db.contacts.find().forEach(printjson);
'
{
  _id: ObjectId('6aeb4483ff19c29b8b07c522'),
  name: 'Nandan. K. R',
  email: 'nandanmayura600@gmail.com',
  subject: 'test',
  message: 'testing multi container\n',
  createdAt: ISODate('2026-11-04T14:43:15.613Z'),
  updatedAt: ISODate('2026-11-04T14:43:15.613Z'),
  __v: 0
}
{
  _id: ObjectId('6aeef20467390c18ba199ae0'),
  name: 'user2',
  email: 'user2@gmailcom',
  subject: 'devops',
  message: 'testing multi container',
  createdAt: ISODate('2026-11-07T09:40:20.755Z'),
  updatedAt: ISODate('2026-11-07T09:40:20.755Z'),
  __v: 0
}
```

This provides end-to-end proof that the backup and recovery process works successfully.

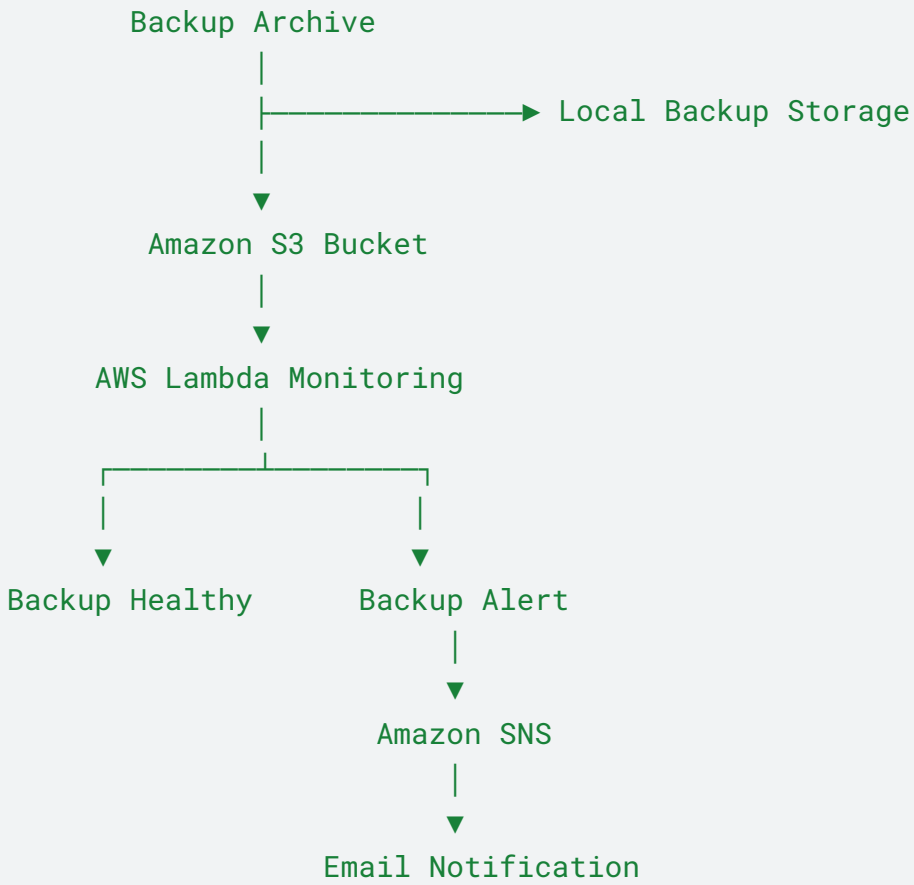
---

23. Complete Backup and Recovery Workflow

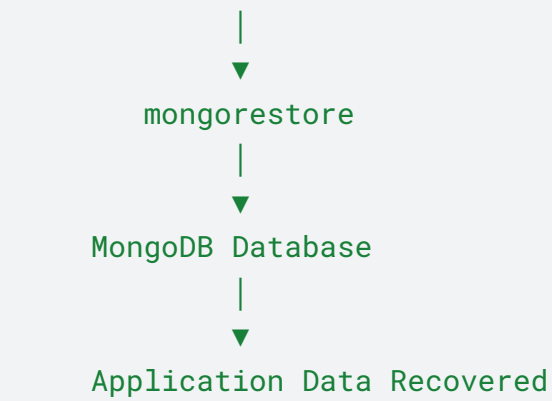
The final data protection architecture can be summarized as follows:

None

```
Dockerized Contact Application
            |
            ▼
      MongoDB Database
            |
            ▼
        mongodump
            |
            ▼
```



Disaster Recovery:



24. Key Results

The following results were successfully achieved:

- Docker Compose application deployed successfully.
- Frontend, backend, and MongoDB containers were running.
- The application was accessible externally through Amazon EC2.
- Contact data was successfully stored in MongoDB.
- Custom Docker networks were implemented.
- Frontend and database layers were network-segmented.
- MongoDB data persisted on the EC2 host.
- MongoDB backups were successfully created.
- Backup archives were successfully uploaded to Amazon S3.
- Daily backup automation was configured using Cron.
- Local backup retention cleanup was implemented.
- AWS Lambda successfully detected an outdated backup.
- Amazon SNS successfully delivered a backup alert.
- MongoDB data was successfully restored from a backup archive.
- Restored contact data was verified successfully.

---

25. Key DevOps Concepts Demonstrated

This project demonstrates practical experience with the following DevOps concepts:

Containerization

- Docker images
- Docker containers
- Multi-container applications
- Docker Compose

Docker Networking

- Custom bridge networks
- Container isolation
- Service-to-service communication

Data Persistence

- Bind mounts

- Persistent database storage
- Container-independent data

Cloud Integration

- Amazon EC2
- Amazon S3
- AWS Lambda
- Amazon SNS
- IAM-based AWS access

Automation

- Cron scheduling
- Automated database backups
- Backup retention

Monitoring and Alerting

- Backup age validation
- Serverless monitoring
- Automated SNS notifications

Disaster Recovery

- MongoDB backup creation
- Data deletion testing
- Database restoration
- Post-recovery verification

---

26. Conclusion

This project successfully implemented a complete containerized application deployment and data protection workflow.

The Contact Application was deployed using Docker Compose with separate frontend, backend, and MongoDB containers. Docker network segmentation was implemented to control communication between application components, while MongoDB data was persisted using an EC2 host bind mount.

An automated backup system was implemented to create MongoDB backup archives and upload them to Amazon S3. Cron scheduling ensured that backups could run automatically without manual intervention, while retention cleanup helped manage local backup storage.

The project also implemented monitoring using AWS Lambda to detect outdated or missing backups. When an alert condition occurred, Amazon SNS successfully delivered an email notification.

Finally, disaster recovery testing demonstrated that MongoDB data could be successfully restored from a backup archive. The restored contact records were verified directly inside MongoDB.

Overall, this project demonstrates a practical implementation of key DevOps principles including:

- Containerization
- Networking
- Persistent storage
- Automation
- Cloud storage
- Monitoring
- Alerting
- Disaster recovery

The final solution provides a strong example of how a containerized application can be deployed and operated with automated data protection and recovery capabilities.

Final Result: The Dockerized Contact Application was successfully deployed, monitored, backed up, and tested for disaster recovery.